

LAM Project 2024

Federico Montori, Nicolas Lazzari

April 2024

Rules

In this document we describe two possible projects for the exam of “Laboratorio di applicazioni mobili” course. The project can be implemented by **groups of maximum 2 people** (individual projects are obviously allowed as well), provided they implement the project integration. Each student/group can choose to develop one of the projects proposed here (valid until February 2025) or suggest something else based on his/her personal interests. In the latter case, project proposals **must be individual only** and should be submitted via e-mail to Dr. Nicolas Lazzari `nicolas.lazzari3@unibo.it`, with a brief description of the application goals, contents, and a list of **all** the features that you propose to implement. In extremely exceptional cases we can allow group projects outside the one proposed here, but they will need to be discussed with us in advance. The use of Git (or any other versioning system) is strongly encouraged. The following project description contains a minimum set of requirements for the application. Students are strongly encouraged to expand the track, by adding new features to the application, and/or further customizing the contents. Any questions regarding the projects are to be asked to Dr. Nicolas Lazzari via e-mail (`nicolas.lazzari3@unibo.it`). Regardless of the choice, every student/group is required to produce:

1. *One or more mobile applications*, there is no constraint on the language and the framework used: it can be native or hybrid, but it cannot be a Web Application (*i.e.* it must run natively on the device, and there must be a reason to do so).
2. *A project report*, which is a document that describes the application produced, focusing on the workflow and the design choices. In particular, the Report should be named `SURNAME1_SURNAME2.pdf` and contain:
 - The name, surname, email, and matriculation number of each component of the group.
 - Overview of the application with screenshots.
 - Implementation details on how you chose to implement the functionalities.

A good report is probably between 10 and 15 pages. Less than 5 pages are probably bad, and more than 15 is probably too many. The quality of the report **WILL** be part of the evaluation. **A good report also contains about 70% of implementation details and choices and only the remaining 30% of screenshots, overview...**

3. *A presentation*, which consists of a set of slides that will help you in the project discussion. They should contain a brief recap of the report alongside screenshots of the application. We suggest producing around 10 - 15 slides since the presentation time is approximately 10 minutes (a group discussion may last longer). Furthermore, **each** component of the group must know the details of the entire implementation and will be possibly asked about it during the project discussion.

The **CODE** of the mobile application and the **REPORT** have to be uploaded exclusively on the Virtuale platform in the dedicated section¹ (there are 6 deadlines throughout the year). They must be enclosed in a single .zip file named **SURNAME1.SURNAME2.zip**. In the case of a group, a single component is in charge of completing the upload. If the archive is too big for Virtuale, you can remove the directory “build” from your code (Android only). If the archive is still too big, then the student(s) must share the zipped code via Drive/OneDrive, etc. In this case, the students must still upload on Virtuale a zip file containing the Report, the link to the shared code, and its **hash**, so that we can prove that it was delivered on time. Projects delivered via e-mail will **NOT** be taken into account. The **SLIDES**, instead, must be brought along on the day of the oral examination.

Do not forget that the oral examination consists also in a theoretical part, which is **individual**. Therefore, knowledge of the topics discussed in class (both iOS and Android) is required. The exam consists of the project discussion (which is per group) and the oral examination (which is per student) and **must** be booked via Almaesami.

The Project: “Personal Physical Tracker”

Overview

In the following project, the student is required to implement an interactive application that monitors the physical activity of the user depending on the mean of transportation. A user can be walking, staying still, or, for example, driving a car (there are many other activities that can be added). The goal of the project is to display the user with an interface where he/she can start/stop one of these activities and these will be recorded in a local database. The user can get access to all the activities done by seeing them in a list (or better, a calendar) and receive the report of the monthly activities through plots (e.g. pie charts). Furthermore, when the user is walking/running, we also want to

¹<https://virtuale.unibo.it/course/view.php?id=28374>

record his/her number of steps taken per activity chunk, so that we know how many steps one person has made daily. The app also provides the user with a background functionality, either to record the amount of time the user has been in a certain area, or performing the activity recognition in the background. Disclaimer: the term “activity” in this section means a physical activity, not an Android Activity.

Record the activities

The app should be able to display an interface with which the user can start/stop an activity. The app must record the time spent within this activity and, once the activity is done, the app must be able to save the information in a local database. An activity is of type: walking, driving a car, or sitting. The types of activities can be increased, but these are mandatory. Activity types can have special features attached to them. For the activity type “walking” it is mandatory to record the number of steps as a special feature. This can be done in Android using: <https://developer.android.com/health-and-fitness/guides/basic-fitness-app/read-step-count-data>. The user must be able to see the history of past activities either in a list or in a calendar (or any other means of visualization) and there must be at least one filter in the visualization. Logically, it is nearly impossible to cover all 24 hours every day with activities, even though this would be the ultimate goal of the application. The student must then find a way to handle the absence of activities on certain time frames over the day (they can be simply additional activities of type “unknown”, for example).

Display Charts

The app must have a section that shows at least two different charts about the activities. For instance, there can be a pie chart showing the activity types over the past month and how much they have been performed. Another idea can be a line plot showing the daily number of steps taken in the past month.

Perform background jobs

The application must be able first of all to send periodical notifications to the user, reminding him/her that it is time to record activities, or that it is time to do more steps because today the user is looking kinda static... There needs to be at least one periodic notification. Furthermore, the application also must do **one of** these background operations:

- The app must understand in the background what kind of activity the user is performing and then record it autonomously without the user having to insert it manually. One way to do this in Android could be using the Activity Recognition API “<https://developer.android.com/develop/sensors-and-location/location/transitions>”. Maybe it would be nice to ask the user if the guess is correct from time to time.

- Alternatively, the app must record the user's presence within a certain area of interest that must be registered in advance. This can be done by setting Geofences <https://developer.android.com/develop/sensors-and-location/location/geofencing>. Note that these additional recordings are not activity types, they are separate concepts that have to be handled differently (the student must choose how).

Project Integration for 2 people

In the case of a project for 2 people, students must implement also the following additional feature: the application should be able to display data coming from other users. In particular, a user can contact other users with the same app and “follow” them. More in detail, the application has to store, process, and visualize the other followed users' data, while keeping each data source separate in memory. For the first two tasks, we suggest using different databases/tables, while for the last one, it is possible to show the data of a different user on a different calendar/list... Data must be saved locally as it is imported, however, an additional good practice could be also “monitoring” the followed user over time if there is a shared repository somewhere or if the data from such a user gets imported again. Logically, we cannot add activities to other users' records, we can only visualize them. Users can share their data in different ways:

- *Remote cloud sync.* Users can use a cloud-shared database to synchronize their data, like Firebase. The sync action can be triggered manually or automatically following custom strategies, and the data schema used must be customized by the students. This does **not** replace the local database for his/her own data.
- *Import/Export of a database dump, using a custom data format.* Users can share the file containing their data through an external app and import it into the application to process the additional data. The schema used for exporting the data must be defined by the students.
- *Proximity share.* Users can share their data when approaching a proximity area with another user. The proximity area can be created using, for instance, BLE beacons. If two users become visible in the same area, they are notified that someone else is available and they can use an M2M communication (WIFI-direct, Bluetooth) to share their data. The data schema used must be customized by the students and the range of the area can be customized through the application settings.

All the previous strategies are valid for the exam evaluation, but (from the top to the bottom) the difficulty increases and so does their weight for the final evaluation.

The Project: “Your city is listening to”

Overview

In the following project, the student is required to implement an interactive application that crowd-sources the music that is played throughout its city. The app will allow its users to authenticate and record audio files from the smartphone’s microphone. Each audio will be geo-tagged and sent to a back-end service, NOT developed by the student, that will answer back with details on the audio that has just being sent. The details received on an audio include information such as the music genre, the instruments detected, and so on. A user will be able to see the audios uploaded by other users in the map of its city and interact with it.

The endpoint of the back-end is described throughout the document, alongside examples of requests using *cURL*². Each endpoint that might answer with code 400 requires authentication, which needs to be supplied through the header **Authentication** using **Bearer <token>**. Description on how to obtain the token are in the section below. The app **MUST** implement the features listed below to be sufficient. Adding features is strongly encouraged and has good repercussions in the evaluation. A project that follows the specifications to its minimum but has no extra feature cannot reach high grades.

Authentication

The app has to interact with the provided back-end to allow its user to sign-up. The user must be able to log-out from the system and log-in again by using the password created during the sign-up phase. A description of the REST endpoint responsible for the signup phase is provided below in Table 1.

Endpoint	/auth
Method	POST
Description	Create a new user.

Example request	
<pre>curl -X 'POST' http://130.136.2.83/lam2024/auth -d '{"username": "aaa", "password": "bbb"}'</pre>	

Status code	400 (<i>User already exists</i>)
Response	{ "detail": "Username already registered" }
Status code	200 (<i>User correctly signed up</i>)
Response	{ "username": "aaa", "id": 10 }

Table 1: Description of the authentication endpoint.

The app will have to obtain a token that needs to be used in order to access the endpoints related to audio upload and querying. The description for the token endpoint is Table 2.

²Students can use <https://curlconverter.com> to convert the example to Java, Kotlin or the language chosen.

Endpoint	/auth/token
Method	POST
Description	Obtain the authentication token.
Example request	
<pre>curl -X 'POST' http://130.136.2.83/lam2024/auth -H "Content-Type: application/x-www-form-urlencoded" -d "username=aaa&password=aaa"</pre>	
Status code	400 (<i>Incorrect credentials</i>)
Response	{ "detail": "Incorrect username or password" }
Status code	200 (<i>User correctly signed up</i>)
Response	{ "client_id": 10, "client_secret": "token" }

Table 2: Description of the token endpoint.

A user can delete its account using the API in Table 3. When an account is deleted, all the songs that it has uploaded are removed.

Endpoint	/auth/unsubscribe
Method	DELETE
Description	Delete a user from the back-end.
Example request	
<pre>curl -X 'DELETE' http://130.136.2.83/lam2024/auth }</pre>	
Status code	400 (<i>Incorrect credentials</i>)
Response	{ "detail": "Incorrect username or password" }
Status code	200 (<i>User correctly removed</i>)
Response	{ "detail": "User deleted" }

Table 3: Description of the endpoint to delete a user.

Record and upload audio

The user must be able to record an audio through the use of the microphone of its phone, similarly to Shazam ³. Before proceeding with the upload, the user must be able to check the audio s/he recorded and repeat the process, if needed. If the user is connected through a mobile network, it must be possible to postpone the upload once a Wi-Fi connection is available. The user will receive a notification once the connection is available and the upload can be

³<https://www.shazam.com/>

performed cheaply and safely. A description of the REST endpoint responsible for the audio upload is provided in Table 6.

Endpoint	/upload
Method	POST
Description	Upload an audio.
Example request	
<pre>curl -X 'POST' 'http://130.136.2.83/lam2024/upload?longitude=43&latitude=44' -F 'file=@audio.mp3;type=audio/mpeg'</pre>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	413 (<i>File is too big</i>)
Response	{ "detail": "File is larger than 2 megabytes." }
Status code	415 (<i>File is not an audio</i>)
Response	{ "detail": "File is not a supported audio file." }
Status code	200 (<i>File correctly uploaded.</i>)
Response	See Table 5 for the features provided.

Table 4: Description of the upload endpoint.

The response received contains whether the upload has been performed successfully and information on the audio content (described in Table 5). The app must save locally this information and the user must be able to access it in order to see its contributions, even without a connection.

Management of uploaded songs

The user must be able to see the songs s/he uploaded through the app, see the information the backend extracted on each song and interact with each song by hiding it from other users or displaying it again. The information must be saved locally and the user must be able to inspect them at will even without a connection. Note that once a song is uploaded, it will not be possible to download them from the back-end. The app must save the audio file locally, so that the user can listen them again. To obtain all the songs uploaded by a user, the app must use the API of Table 6.

A user can hide one of its uploaded songs by using the API in Table 7 and show it using the API in Table 8.

Finally, a user can delete a song, which will be completely erased from the backend, by using the endpoint of Table 9.

Map

Table 10 describes the API to obtain all the songs uploaded by other users and their location. The app must display each song in a map. Note that the

endpoint does not return any of the tags associated with a song. The app must obtain those information using the API of Table 11. Since many songs might be available in the database of the backend, the app **MUST NOT** fetch the details of every song in the database. Instead, the user must be able to click in a marker and see its details. An optional extension, which allows achieving a better grade, it to automatically fetch the details of each marker once a zoom level is reached. For example, if the user zooms into the maps and sees only 5 markers, then the app automatically retrieves the information for each song displayed in the map. In that case, the user can filter the songs based on the features of Table 5. For example, only visualizing the locations for *dance* music.

Project Integration for 2 people

In the case of a project for 2 people, students must implement also the following additional feature: it must be possible for the user to find the places that for which a recording can be made along a path. For example, if the user has to go from *Piazza Maggiore* to *Porta San Donato*, s/he must be able to use the app to find places along the path to *San Donato* that involve locations (e.g. bars, stores, etc. The students can freely choose **at least 3** types of locations) for which a recording can be made. The user can choose which locations to visit, which will be shown on the map alongside the path to take. Places that do not have any audio geo-tagged in that location should be prioritized and recommended to the user.

In order to find the path and the places along it, there are two possibilities:

- Rely on Google Maps APIs. In particular, by using the Routes API ⁴ it is possible to find the path from two locations while the through the Places API ⁵ is possible to retrieve relevant locations (bars etc.) along it. Note that Maps API are not free ⁶. It is possible to obtain 200\$ worth of credits for free each month, which are sufficient for the sake of the exam. Nonetheless, students that choose to use Google's APIs are expected to properly handle the credits, for example by limiting the number of requests that can be done daily or by displaying to the user the number of remaining searches that can be done for free.
- Rely on OpenStreetMaps (OSM). To overcome the payment to Google, students can rely on OSM, which is a free and open source map effort. Albeit no costs are involved, OSM APIs are less easy to use when compared to Google's ones. The official documentation provides a set of APIs that can be used ⁷ but they might not be sufficient to find paths and relevant places. Students can rely on other free services that work on top of OSM to provide those features ⁸. For example, the Google's Route API can

⁴<https://developers.google.com/maps/documentation/routes/overview>

⁵<https://developers.google.com/maps/documentation/places/web-service/overview>

⁶<https://mapsplatform.google.com/pricing/>

⁷https://wiki.openstreetmap.org/wiki/API_v0.6#

⁸See https://wiki.openstreetmap.org/wiki/List_of_OSM-based_services for a complete list

be replaced by using OpenRouteService ⁹, where an API key (with little limitation) can be obtained for free. Similarly, Google's Places API can be replaced by using Overpass Turbo ¹⁰.

Both Google Maps and OSM are equivalent for the sake of the exams. In both approaches, students have to autonomously obtain access to the API and figure out how to use them. Regardless of the choice, it is important that during the exam it will be possible to test the feature.

⁹<https://openrouteservice.org/>

¹⁰<https://overpass-turbo.eu/>

JSON field	Description
bpm	An integer which are the estimated Beats per minutes of the song.
danceability	A score in $[0, 1]$ which estimates the danceability of the song.
loudness	A score which measures how loud is the recorded song.
mood	Contains another JSON object which provides a probability of the song being in a specific mood. The possible moods are: action, adventure, advertising, background, ballad, calm, children, christmas, commercial, cool, corporate, dark, deep, documentary, drama, dramatic, dream, emotional, energetic, epic, fast, film, fun, funny, game, groovy, happy, heavy, holiday, hopeful, inspiring, love, meditative, melancholic, melodic, motivational, movie, nature, party, positive, powerful, relaxing, retro, romantic, sad, sexy, slow, soft, soundscape, space, sport, summer, trailer, travel, upbeat, uplifting.
genre	Contains another JSON object which provides a probability of the song being in a specific genre. The possible genre are: 60s, 70s, 80s, 90s, acidjazz, alternative, alternativerock, ambient, atmospheric, blues, bluesrock, bossanova, breakbeat, celtic, chanson, chillout, choir, classical, classicrock, club, contemporary, country, dance, darkambient, darkwave, deephouse, disco, downtempo, drumnbass, dub, dubstep, easylistening, edm, electronic, electronica, electropop, ethno, eurodance, experimental, folk, funk, fusion, groove, grunge, hard, hardrock, hiphop, house, idm, improvisation, indie, industrial, instrumentpop, instrumentalrock, jazz, jazzfusion, latin, lounge, medieval, metal, minimal, newage, newwave, orchestral, pop, popfolk, poprock, postrock, progressive, psychedelic, punkrock, rap, reggae, rnb, rock, rocknroll, singersongwriter, soul, soundtrack, swing, symphonic, synthpop, techno, trance, triphop, world, worldfusion.
instrument	Contains another JSON object which provides a probability of the song using a specific instrument. The possible genre are: accordion, acousticbassguitar, acousticguitar, bass, beat, bell, bongo, brass, cello, clarinet, classicalguitar, computer, doublebass, drummachine, drums, electricguitar, electricpiano, flute, guitar, harmonica, harp, horn, keyboard, oboe, orchestra, organ, pad, percussion, piano, pipeorgan, rhodes, sampler, saxophone, strings, synthesizer, trombone, trumpet, viola, violin, voice.

Table 5: Informations extracted on the audio uploaded.

Endpoint	/audio/my
Method	GET
Description	Obtain the songs uploaded by the authenticated user.
Example request	
<code>curl -X 'GET' http://130.136.2.83/lam2024/audio/my</code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	200 (<i>File correctly uploaded.</i>)
Response	{ "longitude": float, "longitude": float, "id": int, "hidden": bool }

Table 6: Description of the endpoint to obtain own songs.

Endpoint	/audio/my/<song_id>/hide
Method	GET
Description	Hide the song from other users.
Example request	
<code>curl -X 'GET' http://130.136.2.83/lam2024/audio/my/<song_id>/hide</code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	401 (<i>Audio not found</i>)
Response	{ "detail": "Audio not found" }
Status code	200 (<i>Song successfully hidden</i>)
Response	{ "detail": "Audio hidden" }

Table 7: Description of the endpoint to hide own songs.

Endpoint	/audio/my/<song_id>/show
Method	GET
Description	Show the song to other users.
Example request	
<code>curl -X 'GET' http://130.136.2.83/lam2024/audio/my/<song_id>/show</code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	401 (<i>Audio not found</i>)
Response	{ "detail": "Audio not found" }
Status code	200 (<i>Song successfully hidden</i>)
Response	{ "longitude": float, "longitude": float, "id": int, "hidden": bool }

Table 8: Description of the endpoint to show an owned song.

Endpoint	/audio/<song_id>
Method	DELETE
Description	Deletes the song from the backed.
Example request	
<code>curl -X 'DELETE' http://130.136.2.83/lam2024/audio/<song_id></code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	401 (<i>Audio not found</i>)
Response	{ "detail": "Audio not found" }
Status code	401 (<i>User is not authorized to delete the audio</i>)
Response	{ "detail": "Can't delete audio from other users." }
Status code	200 (<i>Song successfully deleted</i>)
Response	{ "detail": "Song deleted" }

Table 9: Description of the endpoint to delete a song from the back-end.

Endpoint	/audio/all
Method	GET
Description	Obtain a list of all the songs in the database.
Example request	
<code>curl -X 'GET' http://130.136.2.83/lam2024/audio/all</code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	200
Response	{ "longitude": float, "longitude": float, "id": int }

Table 10: Description of the endpoint to obtain all the songs.

Endpoint	/audio/<audio id>
Method	GET
Description	Obtain a song from the database using its id.
Example request	
<code>curl -X 'GET' http://130.136.2.83/lam2024/audio/<audio id></code>	
Status code	401 (<i>User is not authorized</i>)
Response	{ "detail": "Not authenticated" }
Status code	200
Response	{ "longitude": float, "longitude": float, "id": int, "creator_id": int, "creator_username": string, "tags": See Table 5 }

Table 11: Description of the endpoint to a single song by its id.